ASE Chapter 3

V3 Doctor's Vocabulary Coverage Pack

Term Block 01: Requirements Engineering Process & Present State of Practice

The Requirements Engineering (RE) Process is the disciplined, structured sequence of activities through which customer requirements are discovered, analyzed, formally specified, and validated. The process takes the Problem Statement as input, produces the SRS as output, and consists of four crucial, tightly coupled iterative activities:

Requirements Elicitation: Uncovers stakeholder expectations, operational realities, and system boundaries through collaborative discovery techniques.
Requirements Analysis: Refines, scrutinizes, and models elicited requirements (DFD leveling, Data Dictionary, ER diagrams) to remove ambiguities and resolve conflicts.
Requirements Documentation (SRS): Formalizes requirements into an unambiguous, verifiable contractual document conforming to IEEE Std 830-1998.
Requirements Review (Validation): Rigorously inspects the draft SRS against quality criteria through peer walkthroughs to catch defects early, driving feedback loops.

(Pre-project gateway: A feasibility study evaluates whether the project is viable before significant resources are committed, preceding the four-step requirements-engineering cycle.)

Present State of Requirements Engineering Practice:

The course materials identify recurring practical difficulties:

Requirements are difficult to uncover.
Requirements can change during development.
Teams may rely too heavily on CASE tools.
Project schedules may be tight.
Communication barriers can obstruct understanding.
Market-driven development and limited resources add pressure.

Source: companion Slide 4; K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1, pp. 40–41.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة الطالب والفخ الامتحاني:**
>
> - لا تسرد مراحل الـSDLC كاملة؛ فخطوات RE المعتمدة في المنهج (شريحة 3 وشكل 3.1) محصورة في: الاستخلاص (Elicitation) ← التحليل (Analysis) ← التوثيق (Documentation) ← المراجعة (Review) مع حلقتي تغذية راجعة. أما دراسة الجدوى فهي تقييم سابق لتحديد جدوى المشروع.
> - Slide 4 يسرد صعوبات الممارسة المذكورة أعلاه؛ لا تحوّل القائمة إلى نتائج استبيان أو ترتيب لأسباب فشل المشاريع.

Sources: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1–§3.2, pp. 40–45; companion Slides 2–5.

Term Block 02: Types of Requirements & Interface Specifications

Software requirements are categorized into distinct conceptual classes:

Functional Requirements:

- **Statements of services, computations, data transformations, and behavioral responses the system must provide.** They define what the system must do in response to specific operational inputs.

Non-Functional Requirements:

- **Constraints placed on the services or functions offered by the system.** These include performance targets (response time, throughput), quality attributes (reliability, availability, security, usability), platform constraints, and regulatory standards.

User Requirements vs. System Requirements:

- **User Requirements:** High-level abstract statements in natural language and informal diagrams intended for clients and end-users to establish contractual scope.
- **System Requirements:** Detailed, structured technical specifications of all software functions and operational constraints used by software architects and developers for implementation.

Interface Specification (Slide 9 Focus):

- **Crucial for customers and system integrators to ensure seamless interoperability.** Encompasses three primary types:
  - **Procedural Interfaces (Application Programming Interfaces - APIs):** Function signatures, parameters, return types, and exceptions.
  - **Data Structures:** Shared in-memory data representations exchanged across system modules.
  - **Representation of Data:** Standard physical encodings, network packet formats, and communication schemas.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة الطالب والفخ الامتحاني:**
>
> - ميز بدقة بين تصنيف سلايد 9 الخاص بـInterface Types (APIs, Data structures, Representation of data) وبين تبويب معيار IEEE 830 الخارجي (User, Hardware, Software, Communications).
> - المتطلبات الوظيفية تجيب عن: *ماذا يفعل النظام؟* بينما غير الوظيفية تجيب عن: *ما هي قيود الأداء والأمان؟*

Sources: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.3, pp. 45–48; companion Slides 6–9.

Term Block 03: Feasibility Study: Purpose and Focus

Purpose of Feasibility Studies:

A Feasibility Study is an early evaluation and analysis of the potential impact, viability, and practicality of a proposed software project.

Economic Motivation (Slide 10 Historical Note): Companion Slide 10 attributes the figures **31% cancelled, 53% over budget by an average of 189%, and 94 restarts per 100 projects** to a historical IBM report. The slide gives no report date or full citation; treat these as historical course-slide figures, not current universal rates.
Core Principle: A feasibility study checks whether the proposed system is technically, economically, and operationally viable, including whether major risks can be surmounted, before the project proceeds further.

Focus of Feasibility Studies (Slides 13–15):

Product Concept Viability: Is the software concept practical, and can a system matching the vision statement be engineered?
Cost and Schedule Realism: What are the current estimated cost and schedule bounds, and how wide is the gap between original targets and realistic estimates?
Business Model Justification: Is the business model economically justified when realistic costs are factored in?
Stability of Scope: Can users and developers agree on a detailed user interface prototype? If not, requirements are not yet stable.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة الطالب والفخ الامتحاني:**
>
> - دراسة الجدوى ليست مضيعة للوقت بل أداة حماية؛ وإلغاء المشروع غير القابل للتطبيق في هذه المرحلة يعتبر نجاحاً هندسياً وإدارياً يحمي المؤسسة من خسائر فادحة.

Sources: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.1, pp. 44–45; companion Slides 10–15.

Term Block 04: Requirements Elicitation: Interviews, Brainstorming, FAST, QFD, & Use Cases

Requirements Elicitation is the proactive discovery, collection, and synthesis of user needs, business workflows, and operational constraints from diverse stakeholders.

1. Interviews (The Foundational Technique):

Mechanism: Direct interpersonal dialogue between systems analysts and domain stakeholders (users, managers, operators) utilizing open-ended and closed questions.
Strengths: Uncovers informal workarounds, builds personal stakeholder rapport, and clarifies complex organizational goals.
Limitations: Subject to cognitive bias; interviewees often omit critical tacit knowledge; extremely time-consuming for large distributed user bases.

2. Brainstorming, FAST, and QFD:

Brainstorming: A group technique for generating and discussing candidate requirements; ideas are collected before the group evaluates and refines them.
FAST (Facilitated Application Specification Technique): A facilitated stakeholder session used to agree on scope, clarify needs, and develop a shared requirements specification.
QFD (Quality Function Deployment): Translates customer needs into prioritized technical requirements, helping teams focus on the product characteristics customers value.

3. The Use Case Approach:

Mechanism: Scenario-based behavioral modeling capturing interactions between external **Actors** and the **System Boundary** to achieve observable user goals.
Stereotype Dependencies:

- **<<include>>: Mandatory, unconditional inclusion** of a shared sub-routine; the base use case cannot complete without it (Base ⟶ Included).
- **<<extend>>: Optional, conditional insertion** of variant behavior triggered only at a defined extension point (Extension ⟶ Base).

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة الطالب والفخ الامتحاني:**
>
> - في المقابلات، أكبر عائق هو المعرفة الضمنية (*Tacit Knowledge*) التي يمارسها الموظف كعادة يومية وينسى ذكرها للمحلل.
> - اتجاه الأسهم في Use Case حاسم في الامتحان: <<include>> يشير للأمام نحو الحالة الفرعية الإلزامية، بينما <<extend>> يشير للخلف عائداً إلى الحالة الأصلية.

Sources: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.3.1 & §3.3.4, pp. 48–61; companion Slides 16–26, 27–50.

Term Block 05: Requirements Analysis: DFDs, Data Dictionaries, ERDs, & Prototyping

Requirements Analysis models the operational problem domain to uncover omissions, structural flaws, and boundary conflicts through four primary techniques:

Data Flow Diagrams (DFDs):

- **Hierarchical behavioral modeling tool depicting data movement and functional transformations.**
- Uses four standard symbols: **Process (Bubble), Data Flow (Arrow), Data Store (Parallel Lines), and External Entity (Rectangle).**
- Operates on strict conservation rules: Level-0 Context Diagram must be **strictly balanced** with Level-1 decompositions. Direct connections between external entities or data stores are illegal.

Data Dictionaries (DD):

- **Structured, centralized repository defining the schema, composition, and constraints of every data item in the system.**
- Employs standard formal algebraic operators:
  - = : **is defined as / composed of**
  - + : **AND / Sequence**
  - [a/b] : **select either a or b**
  - y{a} : **repeat a at least y times**
  - {a}z : **repeat a at most z times**
  - y{a}z : **repeat a from y through z times**
  - (a) : **a is optional**

Entity-Relationship Diagrams (ERDs):

- **Structural data modeling depicting persistent data at rest via Entities, Attributes, and Relationships.**
- Identifies relationship degrees: **Unary (Degree 1), Binary (Degree 2), Ternary (Degree 3), and N-ary.**
- When mapping an M:N relationship into relational tables, a junction table is a common implementation option; a conceptual ER diagram may show the M:N relationship directly.

Software Prototyping (Throw-Away vs. Evolutionary):

- **In large software systems, prototyping should be "Throw-Away".** Developed using "quick and dirty" methods to clarify requirements; discarding the experimental code prevents unmodular, unmaintainable artifacts from contaminating production architecture.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة الطالب والفخ الامتحاني:**
>
> - الـDFD يمثل حركة العمليات، والـERD يمثل بنية البيانات، والقاموس يوحّد معنى عناصر تدفق البيانات.
> - الكتاب يستخدم [a/b] للاختيار، بينما تظهر صيغة مختلفة في Slide 88؛ اعتمدت هنا صيغة الكتاب الأساسي.
> - النموذج التجريبي السريع (*Throw-away prototype*) صُمم لجمع المعرفة والرمي في القمامة؛ ومحاولة بيعه أو تحويله لمنتج حقيقي هي بداية الكارثة البرمجية.

Sources: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5, pp. 61–72; companion Slides 82–91, 116.

Term Block 06: Requirements Documentation: Nature, Characteristics, & IEEE Std 830

1. Nature of the Software Requirements Specification (SRS):

The SRS is the official, formal document that establishes the binding contractual agreement between customer and development organization.
Black-Box Abstraction Principle: It completely specifies external behavior, inputs, outputs, and constraints **without prescribing internal design, algorithms, or code architecture.**
Core Issues Addressed: Functionality, External Interfaces, Performance, Quality Attributes, and Design Constraints.

2. Characteristics of a Good SRS (IEEE Std 830-1998 Criteria):

Correct: Accurately reflects true customer intentions.
Unambiguous: Every requirement statement has exactly one valid interpretation.
Complete: Defines responses to all possible input conditions (valid and invalid).
Consistent: Contains no conflicting requirements or mutually contradictory states.
Ranked for Importance/Stability: Prioritizes features into essential, conditional, and optional.
Verifiable: A finite, cost-effective test or inspection process exists to prove conformance.
Modifiable: Structured modularly so changes do not produce uncontained ripple effects.
Traceable: Every requirement possesses a unique tag supporting forward and backward tracing.

3. Organization of the SRS (IEEE Std 830 3-Tier Structure):

Section 1: Introduction: Purpose, Scope, Definitions/Acronyms, References, and Overview.
Section 2: Overall Description: Product Perspective, Functions Summary, User Characteristics, General Constraints, and Dependencies.
Section 3: Specific Requirements: Detailed verifiable functional rules, interface specifications (User, Hardware, Software, Communications), and performance bounds.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة الطالب والفخ الامتحاني:**
>
> - الشرط الأكثر تكراراً في الامتحانات هو **Verifiable**؛ أي متطلب يحتوي على صفات غير مقاسة مثل "سريع" أو "واجهة سهلة" يفقد صفة التحقق ويعد عيباً في الـSRS.
> - تذكر دائماً الأقسام الثلاثة الرئيسية لمعيار IEEE 830: المقدمة، الوصف العام، والمتطلبات المحددة.

Sources: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6, pp. 72–78; companion Slides 101–110.

Term Block 07: Requirements Validation: The 6-Stage Review Process & Checklists

Requirements Validation ensures that the SRS accurately represents customer expectations and satisfies institutional quality standards before architectural design commences.

1. The 6-Stage Requirements Review Process (Figure 3.21 & Slide 113):

Plan Review: Establish review scope, schedule sessions, and select the multi-disciplinary inspection team.
Distribute SRS Documents: Deliver the draft specification to reviewers in advance.
Read Documents: Reviewers individually inspect the document, noting ambiguities and standard violations.
Organize Review: Convene structured walkthrough to log defects and consensus resolutions.
Follow-Up Actions: Remediate logged issues, consult stakeholders on conflicts, and verify needed changes.
Revise Document: Requirements authors modify the SRS to produce the approved baseline document.

2. Problem Resolution Actions (Slide 114):

Requirements Clarification: Rewrite vague or subjective phrasing.
Missing Information: Re-engage stakeholders to supply omitted operational rules.
Requirements Conflicts: Convene stakeholder negotiations to resolve contradictory demands.
Unrealistic Requirements: Consult management to de-scope technically or economically infeasible items.
Security Issues: Align system requirements with mandatory security standards.

3. The 7-Point Quality Review Checklist (Slide 115):

1. **Redundancy:** Are statements repeated unnecessarily?
2. **Completeness:** Are all inputs, outputs, and operational states defined?
3. **Ambiguity:** Can any requirement statement be interpreted in multiple ways?
4. **Consistency:** Are there conflicting constraints or contradictory states?
5. **Organization:** Does the document strictly conform to the IEEE Std 830 structure?
6. **Conformance:** Does the specification adhere to institutional and legal standards?
7. **Traceability:** Is every requirement uniquely tagged and bidirectionally traceable?

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- مراجعة المتطلبات (*Requirements Review*) عملية جماعية منظمة وليست مجرد قراءة سريعة؛ والخطوة التي تفشل فيها معظم الفرق هي إهمال القراءة الفردية المسبقة (الخطوة 3).
- تسلسل الشريحة 113 يوجه من تنظيم المراجعة (Organize review) إلى إجراءات المتابعة (Follow up actions) ثم تعديل الوثيقة (Revise document). احفظ نقاط الفحص السبعة نصاً كما وردت في سلايد 115.

Sources: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.7, pp. 92–94; companion Slides 112–115.
